

1. Project Overview & Objectives	3
2. System Architecture Diagram	4
3. Directory Structure	5
4. Technology Stack & Dependencies	6
5. Dataset & Data Pipeline	7
5.1 Dataset Details	7
5.2 Data Augmentation Strategy	7
5.3 Preprocessing & Normalisation	8
6. Deep Learning Model Architecture	9
6.1 Why EfficientNetV2B0?	9
6.2 Classification Head	9
6.3 Model Architecture Diagram	9
7. Training Pipeline (train_model.py)	10
7.1 Phase 1 — Head Training	10
7.2 Phase 2 — Fine-Tuning	11
7.3 Class Weight Balancing	11
7.4 Custom SavelfBetter Callback	12
7.5 Post-Training Calibration	12
8. Inference & Web Interface (predict_app.py)	13
8.1 Prediction Logic	13
8.2 Severity Grading System	14
8.3 Gradio Dashboard Design	14
9. CLI Verification Tool (quick_test.py)	15
10. Safety Mechanisms & Design Principles	16
11. Known Issues & Improvement Roadmap	17
12. Setup & Deployment Guide	18

RAILWAY TRACK FAULT DETECTION SYSTEM

End-to-End Deep Learning System for Railway Safety

Architecture:

Class Weight

Interface:

Dataset:

Author:

Version:

EfficientNetV2B0 (Transfer Learning)

TensorFlow 2.18 / Keras

Gradio Web Dashboard

~300 images — Kaggle (salmaneunus)

Souvik SB | IITR Project

2.0 — Improved Fine-Tuning Pipeline

github.com/SOUVIKSB1/Railway_Track_Fault_Detection

■ 1. Project Overview & Objectives

The **Railway Track Fault Detection System** is an end-to-end computer vision pipeline designed to automatically classify images of railway tracks as either **Defective** or **Non_Defective (Safe)**. The system is built to support field maintenance teams by providing a fast, AI-powered second opinion on track inspections — reducing human error and enabling prioritised maintenance scheduling.

Core Objectives

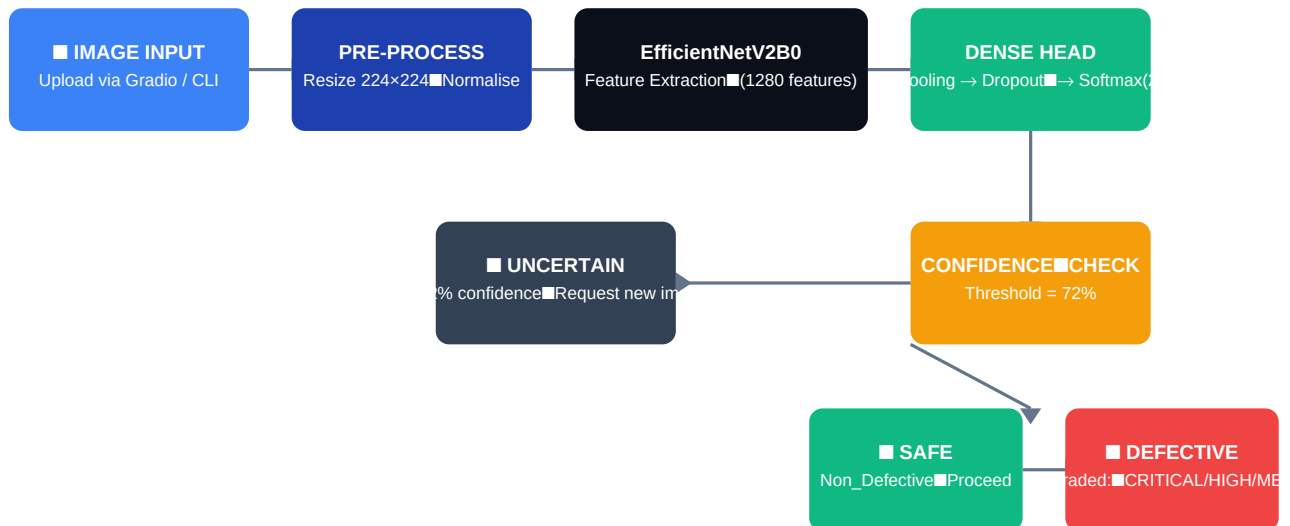
- **Automate visual inspection** — Replace or supplement slow, labour-intensive manual surveys with instant AI analysis.
- **Prioritise safety** — The model is explicitly tuned to minimise missed defect detections (false negatives), since a missed crack is more dangerous than a false alarm.
- **Handle small datasets gracefully** — With only ~300 training images, the system uses transfer learning and aggressive augmentation to generalise well.
- **Provide actionable severity grading** — Not just 'defective/safe', but a 3-level severity system (CRITICAL / HIGH / MEDIUM) that maps directly to operational protocols.
- **Prevent overconfident wrong predictions** — A confidence threshold (72%) ensures the system returns 'Uncertain' for ambiguous inputs rather than forcing a wrong class.

Key Design Decisions

Dataset Size	~300 images — necessitates transfer learning, augmentation, and lightweight backbone
Backbone Choice	EfficientNetV2B0 — ~30% fewer parameters than DenseNet121, reducing overfitting risk on small data
Safety Priority	Defective class recall is the primary metric; overall accuracy is secondary
Confidence Gate	72% threshold before the model commits to any prediction, preventing 'confident nonsense' on irrelevant images
Fine-tuning Guard	Custom SaveIfBetter callback ensures Phase 2 fine-tuning cannot overwrite a Phase 1 checkpoint that performed better
Label Smoothing	10% smoothing prevents the model from targeting absolute 1.0 confidence, reducing overconfidence

2. System Architecture Diagram

The following diagram shows the high-level data flow through the complete system — from raw image acquisition all the way through to the operational decision output.



The system has three possible output states for any given input: **Safe** (green), **Defective** (red, with severity grade), or **Uncertain** (amber) when confidence is insufficient for a reliable decision.

■ 3. Directory Structure

The project is organised under an IITR/ root directory with all active code and outputs contained in the RAILWAY_DEFECT/ workspace.

```

IITR/ (Project Root)
■■■ .gitignore          → Excludes models, datasets, __pycache__
■■■ README.md           → Basic GitHub readme
■■■ CLAUDE_CONTEXT.md   → Claude AI documentation context
■■■ RAILWAY_DEFECT/     → Core workspace
    ■■■ train_model.py   → Training & fine-tuning pipeline
    ■■■ predict_app.py   → Gradio web dashboard
    ■■■ quick_test.py    → CLI single-image test tool
    ■■■ requirements.txt → Python dependencies
    ■■■ class_names.json → ["Defective", "Non_Defective"]
    ■■■ model_metadata.json → Threshold, best_accuracy, image_size
    ■■■ railway_model.keras → Saved trained model (gitignored)
    ■■■ inspection_report.csv → Prediction logs
    ■■■ results/
        ■■■ classification_report.txt
        ■■■ confusion_matrix.png
        ■■■ training_graphs.png
  
```

File Reference Table

File / Directory	Description	Parent
IITR/	Project root	
.gitignore	Excludes models, datasets, and caches from version control	root
README.md	Basic project readme for GitHub	root
CLAUDE_CONTEXT.md	Self-contained Claude documentation context package	root
RAILWAY_DEFECT/	Core workspace directory containing all code and outputs	root
class_names.json	Saved class label list: [Defective, Non_Defective]	RAILWAY_DEFECT/
model_metadata.json	Calibration config, confidence threshold, best accuracy	RAILWAY_DEFECT/
inspection_report.csv	Diagnostic log; appended on every new prediction	RAILWAY_DEFECT/
predict_app.py	Gradio web dashboard — main inference application	RAILWAY_DEFECT/
quick_test.py	CLI single-image prediction tool	RAILWAY_DEFECT/
train_model.py	Full model training and fine-tuning pipeline	RAILWAY_DEFECT/

File / Directory	Description	Parent
requirements.txt	Python package dependencies list	RAILWAY_DEFINITION/
results/classification_report.txt	Per-class precision, recall, F1-score	RAILWAY_DEFINITION/results/
results/confusion_matrix.png	Confusion matrix heatmap plot	RAILWAY_DEFINITION/results/
results/training_graphs.png	Train vs validation accuracy and loss curves	RAILWAY_DEFINITION/results/

4. Technology Stack & Dependencies

Package	Purpose
tensorflow==2.18.0	Core deep learning framework; model training, inference, GPU support
keras	High-level neural network API (bundled with TF 2.x)
gradio>=4.0.0	Web dashboard for image upload and real-time inference display
Pillow	Image loading, resizing, and colour-space conversion (PIL)
numpy	Array operations, prediction post-processing
scikit-learn	Class weight calculation, metrics (classification_report, confusion_matrix)
matplotlib	Training curve and confusion matrix visualisation
seaborn	Enhanced confusion matrix heatmap styling
tensorflow-metal	(macOS only) Apple Silicon GPU acceleration via Metal backend

```
# Install all dependencies:
pip install -r requirements.txt

# requirements.txt contents:
tensorflow==2.18.0
gradio>=4.0.0
numpy
scikit-learn
matplotlib
seaborn
Pillow
# tensorflow-metal ← uncomment on Apple Silicon Mac
```

5. Dataset & Data Pipeline

5.1 Dataset Details

Source	Kaggle — salmaneunus/railway-track-fault-detection
Total Images	~300 images across two classes
Classes	Defective / Non_Defective
Split	80% training / 20% validation (stratified via validation_split=0.2)
Image Format	Various JPEG/PNG; resized to 224 × 224 pixels at load time
Class Balance	Approximately balanced but small size requires class weight augmentation
Challenge	300 images is extremely small for deep learning — transfer learning is essential

5.2 Data Augmentation Strategy

Because the dataset is tiny (~240 training images), data augmentation is critical to prevent overfitting. However, augmentations must be domain-appropriate — railway tracks are *structured* objects, so wild augmentations would generate unrealistic training samples.

Parameter	Value	Rationale
rotation_range	15 degrees	Tracks can be on slight curves; too much breaks spatial coherence
width_shift_range	10%	Minor camera alignment offsets in field deployment
height_shift_range	10%	Camera height variation
zoom_range	15%	Variable camera distance to track
horizontal_flip	Enabled	Left-right symmetry exists in tracks
vertical_flip	Disabled	Tracks do not run upside-down — vertical flip is unrealistic
brightness_range	[0.75, 1.25]	Day/night, shadow, and weather lighting changes
channel_shift_range	10.0	Rust, dust, and colour variation on track surfaces
fill_mode	nearest	Fills empty augmentation borders with nearest pixel value

Step-by-step data loading flow: (1) ImageDataGenerator is instantiated with augmentation parameters and a validation_split=0.2 flag. (2) flow_from_directory() reads images lazily from disk, applies augmentation on-the-fly only for the training split, resizes to 224×224, and one-hot encodes labels automatically. (3) Each batch of 16 images is yielded as a NumPy array to the training loop.

5.3 Preprocessing & Normalisation

EfficientNetV2B0 has **built-in internal normalisation** that scales pixel values from the range [0, 255] to the range expected by the model. The `preprocess_input` function from `tensorflow.keras.applications.efficientnet_v2` handles this automatically when passed as the `preprocessing_function` argument to `ImageDataGenerator`.

■■ KNOWN BUG (Double-Preprocessing)

In the current v2 code, `preprocess_input` is applied via `ImageDataGenerator` AND the backbone may apply its own normalisation again internally. This causes pixel values to be processed twice, collapsing them near zero and degrading model performance. The fix is to NOT pass `preprocess_input` to `ImageDataGenerator` and instead let the model's own internal preprocessing layer handle normalisation.

■ 6. Deep Learning Model Architecture

6.1 Why EfficientNetV2B0?

Parameters	~7.1M — lightweight vs DenseNet121 (~7.9M) and ResNet50 (~25.6M)
ImageNet Accuracy	Top-1: 78.7% — strong pretrained features for visual patterns
Input Size	224 × 224 × 3 (RGB)
Key Feature	Progressive learning & compound scaling — efficient at small scale
Why not DenseNet121?	Tested but overfits ~300 image datasets more aggressively
Why not MobileNetV2?	Also tested; EfficientNetV2B0 consistently gives better recall on Defective class

6.2 Classification Head

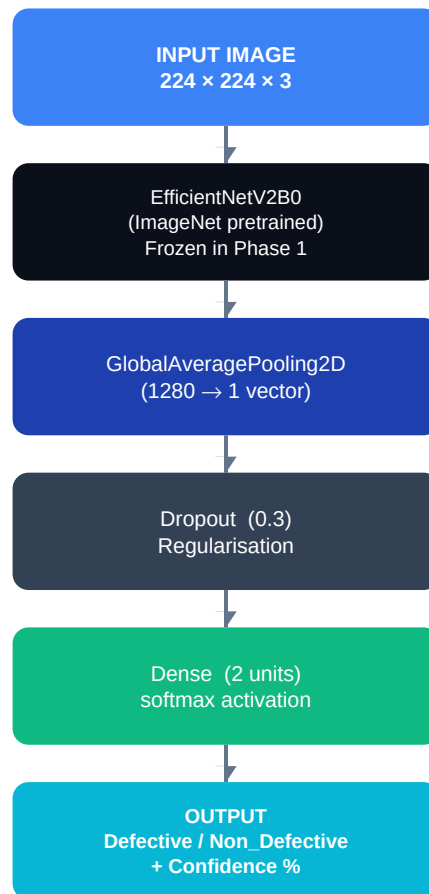
The classification head is attached on top of the frozen EfficientNetV2B0 backbone. It is deliberately minimal to avoid adding too many extra parameters to fine-tune.

```
# Architecture construction (from train_model.py):
base_model = EfficientNetV2B0(
    weights      = 'imagenet',
    include_top= False,          # Remove ImageNet head
    input_shape= (224, 224, 3)
)
base_model.trainable = False    # Frozen in Phase 1

x = GlobalAveragePooling2D()(base_model.output) # 1280 → scalar per channel
x = Dropout(0.3)(x)                       # 30% dropout (regularisation)
output = Dense(2, activation='softmax')(x)    # 2 classes, probabilities sum to 1.0

model = Model(inputs=base_model.input, outputs=output)
```

6.3 Model Architecture Diagram



7. Training Pipeline (train_model.py)

Training is a **two-phase process**. Phase 1 quickly trains only the classification head while the backbone is frozen. Phase 2 carefully unfreezes the top backbone layers with a very low learning rate to adapt ImageNet features to railway track patterns.

Phase	Layers Trained	Learning Rate	Epochs	Purpose
Phase 1 (Head Only)	Dense Head + Pooling only	1e-3 (0.001)	Up to 30 (EarlyStopping)	Learn domain features without disrupting ImageNet weights
Phase 2 (Fine-Tuning)	block5, block6, top layers unfrozen (BatchNorm locked)	1e-4 (0.0001)	Up to 30 (SaveIfBetter)	Adapt deep features to railway domain safely

7.1 Phase 1 — Head-Only Training

In Phase 1, the EfficientNetV2B0 base model is frozen (all its weights are locked). Only the newly-added Dense head is trained. This is safe to do with a relatively high learning rate (1e-3) because there is no risk of corrupting the pretrained backbone weights.

```
# Phase 1: Compile and train head only
model.compile(
    optimizer = Adam(learning_rate=1e-3),
    loss      = CategoricalCrossentropy(label_smoothing=0.10),
    metrics   = ['accuracy']
)

callbacks_phase1 = [
    EarlyStopping(monitor='val_accuracy', patience=7, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4, min_lr=1e-6),
    SaveIfBetter(save_path=MODEL_SAVE) # custom callback
]

history1 = model.fit(
    train_gen,
    validation_data = val_gen,
    epochs          = EPOCHS_HEAD,          # max 30, EarlyStopping may cut short
    class_weight    = class_weights,
    callbacks       = callbacks_phase1
)
```

Label Smoothing (0.10) — Instead of training the model to output exactly [1.0, 0.0] for a 'Defective' sample, label smoothing relaxes the target to [0.90, 0.05]. This prevents the network from becoming overconfident on training data and generalises better to unseen images.

7.2 Phase 2 — Fine-Tuning Top Backbone Layers

After Phase 1 converges, the model has learned to classify tracks using general ImageNet features. Phase 2 refines those features for the specific domain of railway tracks by carefully unfreezing selected deep layers.

```
# Phase 2: Unfreeze top blocks of EfficientNetV2B0
base_model.trainable = True
for layer in base_model.layers:
    # Keep BatchNorm layers frozen – stabilises gradients
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False
    # Only train the top layers (block5, block6, top)
    elif not any(layer.name.startswith(b) for b in ["block5", "block6", "top"]):
        layer.trainable = False

# Re-compile at much lower learning rate to avoid catastrophic forgetting
model.compile(
    optimizer = Adam(learning_rate=1e-4), # 10x smaller than Phase 1
    loss      = CategoricalCrossentropy(label_smoothing=0.10),
    metrics   = ['accuracy']
)

history2 = model.fit(
    train_gen,
    validation_data = val_gen,
    epochs         = EPOCHS_FINE,
    class_weight   = class_weights,
    callbacks      = [SaveIfBetter(save_path=MODEL_SAVE)]
)
```

7.3 Automatic Class Weight Balancing

If the dataset has more Non_Defective than Defective images, the model will naturally bias towards the majority class. Class weight balancing corrects this by penalising misclassifications of the minority class more heavily during training.

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

# Get integer class labels for all training samples
labels = train_gen.classes

# Compute weights: minority class gets higher weight
weights = compute_class_weight(
    class_weight = 'balanced',
    classes      = np.unique(labels),
    y            = labels
)
class_weights = dict(enumerate(weights))
# Example output: {0: 1.14, 1: 0.89} (if Defective is underrepresented)
```

7.4 Custom SavelfBetter Callback

A critical safeguard against Phase 2 regression. The standard Keras ModelCheckpoint callback resets its best-value tracking when a new model.fit() call starts. This means if Phase 1 achieved 88% accuracy but Phase 2 overfits to 80%, the checkpoint would still save the 80% model. **SavelfBetter fixes this** by maintaining a global best-accuracy value across both training phases.

```
class SaveIfBetter(tf.keras.callbacks.Callback):
    def __init__(self, save_path, best_acc=0.0):
        super().__init__()
        self.save_path = save_path
        self.best_acc = best_acc # persists across Phase 1 AND Phase 2

    def on_epoch_end(self, epoch, logs=None):
        val_acc = logs.get('val_accuracy', 0)
        if val_acc > self.best_acc:
            self.best_acc = val_acc
            self.model.save(self.save_path)
            print(f" ■ New best: {val_acc:.4f} → saved to {self.save_path}")
```

7.5 Post-Training Calibration

After training, the system runs calibration to help operators choose the right confidence threshold. It evaluates the model on the validation set at multiple thresholds from 0.50 to 0.90, reporting coverage (% of images above threshold) and accuracy within that covered subset.

The calibration results are saved to `model_metadata.json` and printed as a table. The default threshold of **0.72** was chosen to balance coverage (the model makes a decision for most images) with safety (predictions above this threshold are reliable enough to act on).

■ 8. Inference & Web Interface (predict_app.py)

8.1 Prediction Logic — Step by Step

When a user uploads an image to the Gradio dashboard or calls the `predict_track()` function, the following sequence executes:

Step 1 — Image Loading

Gradio passes the uploaded image as a PIL Image object. The function converts it to RGB (handling greyscale or RGBA inputs) and resizes it to 224×224 pixels.

Step 2 — Array Conversion

PIL Image is converted to a NumPy float32 array via `np.array()`. A batch dimension is added with `np.expand_dims()`, making shape (1, 224, 224, 3) as the model expects.

Step 3 — Preprocessing

The `preprocess_input` function from EfficientNetV2 normalises pixel values from [0, 255] to the range expected by the backbone.

Step 4 — Model Inference

`model.predict(img_array, verbose=0)` returns a (1, 2) array of softmax probabilities, e.g. [0.87, 0.13] = [Defective, Non_Defective].

Step 5 — Confidence Check

`np.max(predictions)` gives the highest confidence. If it is below `CONFIDENCE_THRESHOLD` (0.72), an Uncertain card is returned immediately without classification.

Step 6 — Severity Grading

For Defective predictions: $\geq 92\% \rightarrow \text{CRITICAL}$, $82\text{--}91\% \rightarrow \text{HIGH}$, $72\text{--}81\% \rightarrow \text{MEDIUM}$.

Step 7 — Output

An HTML card (green/red/amber) is returned to the Gradio interface along with a bar chart of class probabilities.

```

def predict_track(img: Image.Image):
    img_rgb = img.convert("RGB").resize(IMAGE_SIZE) # Step 1
    img_array = np.array(img_rgb, dtype=np.float32) # Step 2
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_fn(img_array) # Step 3

    predictions = model.predict(img_array, verbose=0)[0] # Step 4
    max_conf = float(np.max(predictions))
    pred_idx = int(np.argmax(predictions))
    pred_class = CLASS_NAMES[pred_idx]

    if max_conf < CONFIDENCE_THRESHOLD: # Step 5
        return make_uncertain_card(max_conf, CONFIDENCE_THRESHOLD), ...

    if pred_class == "Defective": # Step 6
        severity = "CRITICAL" if max_conf >= 0.92 else "HIGH" if max_conf >= 0.82 else "MEDIUM"
        return make_defective_card(max_conf, severity, note), ...

    return make_safe_card(max_conf), ... # Step 7

```

8.2 Severity Grading System

Severity	Confidence Range	Operational Action	Icon
CRITICAL	>= 92%	HALT TRAFFIC immediately. Emergency maintenance crew dispatch.	■
HIGH	82% – 91%	Alert maintenance team. Impose speed restrictions on the section.	■■
MEDIUM	72% – 81%	Schedule a standard routine inspection within 48 hours.	■
UNCERTAIN	< 72%	Insufficient confidence. Request clearer image with better lighting / angle.	■

8.3 Gradio Dashboard Design

Theme	Dark glassmorphism — #0b0f19 navy background, Orbitron typography
Safe Card	Green rgba(16, 185, 129, 0.1) background — 'TRACK IS SAFE'
Defective Card	Red rgba(239, 68, 68, 0.1) background — 'FAULT DETECTED [SEVERITY]'
Uncertain Card	Amber rgba(245, 158, 11, 0.1) — 'UNCERTAIN DIAGNOSTIC'
Components	gr.Image (upload), gr.Button (trigger), gr.HTML (result card), gr.Label (confidence bars)
Launch	app.launch() — starts local server; share=True for public Gradio link

■ 9. CLI Verification Tool (quick_test.py)

For quick terminal-based testing without launching the full Gradio web interface, `quick_test.py` provides a command-line interface that accepts a single image path and prints the prediction and per-class confidence scores.

Usage

```
# Run a prediction on a single image:
python quick_test.py /path/to/track_image.jpg

# Example output:
# Prediction : Defective (87.3%)
#   Defective      : 87.34%
#   Non_Defective  : 12.66%
```

How It Works

The script loads `model_metadata.json` to retrieve the saved confidence threshold, class names, and image size. It then loads the trained model from `railway_model.keras`, preprocesses the input image identically to the web app, runs inference, and prints results to stdout. It shares the same preprocessing function as the web app, ensuring consistent predictions.

Script Logic Flow

- **Parse arguments** — `sys.argv[1]` provides the image path; exits with usage message if missing.
- **Load metadata** — Reads `model_metadata.json`; falls back to hardcoded defaults if file not found.
- **Load model** — Tries `railway_model.keras` first, then `railway_model.h5` as fallback.
- **Select preprocessor** — EfficientNet preprocessor if architecture contains 'EfficientNet', else DenseNet.
- **Predict** — Opens image → RGB → resize → `np.array` → `expand_dims` → `preprocess` → `model.predict`.
- **Print results** — Displays predicted class, confidence, and all class probabilities.

■ 10. Safety Mechanisms & Design Principles

This system is safety-critical — a missed track defect could lead to a train derailment. Several explicit design decisions reflect this priority.

Safety-First Metric Prioritisation

In a standard classifier, **accuracy** is the headline metric. In this system, **Defective class recall** is the primary goal. Recall measures: "Of all actual defective tracks, what fraction did the model catch?" A high recall (low missed-defect rate) is more important than a high precision (minimising false alarms), because:

- A **False Negative** (missed defect) → train operates on damaged track → potential derailment.
- A **False Positive** (false alarm) → unnecessary inspection stop → minor operational cost.

Confidence Thresholding

Standard classifiers force a binary output even for ambiguous inputs. If someone accidentally uploads a photo of their shoes, the model would assign it to Defective or Safe with some arbitrary confidence. The 72% threshold gate prevents this: any prediction below 72% confidence returns 'Uncertain' rather than a potentially misleading class assignment.

Phase Protection (SavelfBetter)

Fine-tuning deep networks on small datasets is inherently risky — Phase 2 can easily overfit and reduce validation accuracy below Phase 1. The SavelfBetter callback maintains a global best-accuracy tracker that persists across both training phases, ensuring the saved model always represents the best checkpoint ever seen.

BatchNorm Lock During Fine-Tuning

Batch Normalisation layers accumulate running statistics (mean and variance) during training on the large ImageNet dataset. If these are updated with the small railway track dataset in Phase 2, the statistics become corrupted and training destabilises. All BatchNorm layers are kept frozen (trainable=False) throughout Phase 2.

Label Smoothing

With only ~240 training samples, a model can easily memorise the training set and output extreme confidences (99%+) that don't reflect true uncertainty. Label smoothing at 0.10 trains the model to target 0.90 (not 1.00) for the correct class, which produces better-calibrated confidence scores on unseen images.

11. Known Issues & Improvement Roadmap

Current Known Issues

BUG #1 — Double Preprocessing Conflict

preprocess_input is passed to ImageDataGenerator as preprocessing_function, AND EfficientNetV2B0 may apply its own internal normalisation. This double-processing collapses pixel values near zero and significantly degrades model input quality. FIX: Remove preprocessing_function from ImageDataGenerator and rely solely on the backbone's internal rescaling layer.

BUG #2 — Defective Class Prediction Bias

The model consistently biases toward Non_Defective predictions, resulting in low recall (~55%) on the Defective class — the safety-critical class. Root cause is likely the double-preprocessing bug above corrupting input signal. Additional fix: Increase class weight for Defective class beyond balanced weighting (e.g. 2.0x weight for Defective), and consider using focal loss instead of cross-entropy to focus training on hard Defective examples.

Improvement Roadmap

Improvement	Priority	Details
Fix double-preprocessing	High	Remove preprocess_input from ImageDataGenerator; let backbone normalise internally
Increase Defective weight	High	Set class_weight={0: 2.0, 1: 1.0} to strongly penalise missed defects
Focal loss	Medium	Replace cross-entropy with focal loss to focus on hard defective examples
Expand dataset	High	Add more defective track images via web scraping or synthetic augmentation
Grad-CAM visualisation	Medium	Add heatmap overlay showing which track region triggered the defect prediction
Model quantisation	Low	TensorFlow Lite conversion for edge deployment on Raspberry Pi / Jetson
Hugging Face deployment	Medium	Deploy predict_app.py to HF Spaces (app.py + requirements.txt already prepared)
REST API endpoint	Low	FastAPI wrapper around predict_track() for integration with SCADA systems

■ 12. Setup & Deployment Guide

12.1 Local Setup (Step by Step)

Step 1 — Clone the repository

```
git clone git@github.com:SOUVIKSB1/Railway_Track_Fault_Detection.git
cd Railway_Track_Fault_Detection
```

Step 2 — Create Python virtual environment

```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
```

Step 3 — Install dependencies

```
pip install -r RAILWAY_DEFECT/requirements.txt
```

Step 4 — Download dataset from Kaggle

```
kaggle datasets download -d salmaneunus/railway-track-fault-detection
unzip railway-track-fault-detection.zip -d RAILWAY_DEFECT/railway_fault_detector/dataset
```

Step 5 — Train the model

```
cd RAILWAY_DEFECT
python train_model.py
# Output: railway_model.keras, model_metadata.json, results/
```

Step 6 — Launch the web dashboard

```
python predict_app.py
# Opens: http://127.0.0.1:7860
```

Step 7 — CLI quick test

```
python quick_test.py /path/to/test_image.jpg
```

12.2 Hugging Face Spaces Deployment

A deployment-ready app.py and requirements.txt have already been generated for Hugging Face Spaces. To deploy:

```
# 1. Create a new Space at huggingface.co/new-space
#    Select SDK: Gradio

# 2. Push project files:
git clone https://huggingface.co/spaces/YOUR_USERNAME/railway-fault-detector
cp app.py railway-fault-detector/
cp requirements.txt railway-fault-detector/
cp railway_model.keras railway-fault-detector/
cp model_metadata.json railway-fault-detector/
cp class_names.json railway-fault-detector/

# 3. Commit and push
cd railway-fault-detector
git add . && git commit -m "Deploy railway fault detector"
git push
# HF Spaces auto-builds and deploys – public URL in ~2 minutes
```

12.3 Key File Responsibilities Summary

train_model.py	Run once. Trains EfficientNetV2B0, saves railway_model.keras and model_metadata.json
predict_app.py	Run to start web dashboard. Loads saved model, serves Gradio UI at localhost:7860
quick_test.py	Run for CLI testing. Accepts image path, prints prediction without starting a server
model_metadata.json	Written by train_model.py. Read by predict_app.py and quick_test.py at startup
class_names.json	Maps integer indices to class names: {0: Defective, 1: Non_Defective}
requirements.txt	pip install -r requirements.txt installs all dependencies